Pre-Trained Hyper-Hyperheuristics for Optimisation Tasks

Sandy Hay

Student ID: 730023810

Abstract

Pre-trained hyper-heuristic is a concept that formally explored before in the field of optimisation tasks. However, there are plenty of separate studies and papers on the work of high-level strategy hyper-heuristics and the work of pre-training models in the topic of optimisation. This paper aims to combine the two to answer the question of whether it is feasible to create a pre-trained hyper-heuristic, and if so, present the change of model performance and the effectiveness at finding the solution. This literature review will cover the fundamentals of hyper-heuristics and pre-trained models and relevant literature of hyper-heuristics and pre-trained models used in optimisation tasks. Additionally, this paper will feature the specifications and plans for the making of this project.

☑ **I certify that all material in this dissertation which is not my own has been identified.**

Signature: _____

1 Introduction

This project is about pre-trained Hyper-heuristics in optimisation tasks and uncovering the effectiveness of transferability of one or more hyper-heuristics in multiple optimisation scenarios. Hyper-heuristics are high-level strategies that generate or choose from existing low-level heuristics to "solve" difficult search or optimisation problems [1], by selecting the most optimal heuristic to obtain the global optima.

For the origin of the term "hyper-heuristics", this study [2] claims that it first originated over 25 years ago by Denzinger et Al. [3]. However, another study [4] claims that the term was used in a different context in [5]. Hyper-heuristics learn how to optimise problem classes or domains, therefore transfer learning leverages the knowledge obtained to similar problems. This removes the need to start from scratch, and reduces the computation demand for problem domains or large datasets.

Optimization, by definition, is the problem of finding the best solution from all feasible solutions within certain constraints or restrictions, and in this study, the most efficient solution. Classical optimisation methods between 1940-1970s were distinguished by three characteristics: local optimisations, sequential search, and reliance on higher-order derivatives [6]. These approaches slowly evolved into the second generation of optimization algorithms, the heuristic and meta-heuristic algorithms. One of the more common types of optimisation problems are Scheduling Problems (SPs). SPs involve allocating limited resources to tasks over time, with the goal to optimize one or more objectives, for example: minimizing overall completion time, reducing costs or maximizing efficiency [2]. However, the same paper proposes that there are three ways to tackle SPs and optimization problems: exact algorithms, approximate algorithms and heuristic algorithms [2].

Exact algorithms typically find the optimal solution for a problem, with some examples of exact algorithms being depth-first search and breadth-first search [7] [8]. However, as the problem grows (requires more resources, more nodes or more tasks) the computational time to find an exact solution becomes intractable and the algorithm becomes infeasible. For approximate algorithms, they can find a possible solution in polynomial-time computation [9], as studies that show similar regard to these [10] [11] [12] showcase how approximation algorithms can be used in optimisation problems. However, they also suffer the same effect as exact algorithms when the problem scales too large, but not as drastic as exact algorithms with problems that are NP-hard. This creates a need for heuristic algorithms due to their simplicity [2] [13] and scalability for larger optimisation problems, but not all heuristics provide solutions close to an optimum (optimal value).

The rest of this literature review is structured as follows: section [II] will encompass the purpose of heuristics, metaheuristics and hyper-heuristics and how effective they have been in tackling optimization problems, section [III] will deep dive into pre-trained models and transfer learning in optimisation, section [IV] will be a comparative analysis for pre-trained hyper-heuristics, section [V] will be the project specification and the plans I have for the project and section [VI] will finalise with a conclusion, summarizing the review and future research paths.

The rest of this document will include the project specification of this research and the plan to execute this project, with references to the research afterwards.

2 Preliminary Research

2.1 Heuristics, Metaheuristics and Hyper-heuristics

This section expresses the fundamental concepts of hyper-heuristics, their functionality and their effectiveness in tackling optimization tasks.

2.1.1 Heuristics

Heuristics are low-level search methods that are problem-specific strategies that find feasible solutions in a short period of time. Their strength comes from their simplicity and efficiency at solving

optimization tasks. Some sources claim that heuristic algorithms are included in the category of approximation algorithms [14] [15] [16], however, as mentioned in this study [2], heuristic algorithms are not guaranteed to provide a solution close to the optima. This is because heuristic algorithms are designed for efficiency and practicality and not theoretical quality. Therefore, we shall keep them in separate categories.

Heuristic algorithms have two main categories: constructive and perturbative algorithms. Constructive heuristics rapidly build a solution from scratch [17], which are commonly used in time-critical systems, to create solutions for more complex solvers [18]. Constructive heuristics have been used in a variety of domains such as personnel scheduling, job-shop scheduling [19], bin-packing problem [20] [21]. On the other hand, perturbative heuristics modify an already existing solution [17] [22]. These heuristics typically focus on small incremental improvements, employing various strategies such as neighbourhood search or local modification strategies (e.g., swap, insertion, or reallocation operators) to gradually enhance the quality of solutions. Therefore, perturbative heuristics are better suited to problems where obtaining a feasible solution is straightforward, but navigating the heuristic search space efficiently still remains a challenge. Due to their complementary nature, both constructive and perturbative heuristics are often combined to make a hybrid-heuristic within hyper-heuristic frameworks to balance exploration and exploitation. This hybrid approach ultimately achieves a more robust and adaptable performance across various optimisation domains.

However, the nature of heuristics is that they are domain-specific, where they exploit the characteristics of the specified optimisation problem, but when the same heuristic is challenged with a different optimisation problem, it may fail to produce a feasible solution. For multiple or heterogeneous optimisation algorithms, heuristics can fail because:

1. The characteristics that make the heuristic effective no longer hold
2. The heuristic has gotten stuck on a poor local optima
3. The cost function's shape or constraints differ in unexpected ways

2.1.2 Metaheuristics

Metaheuristics are high-level algorithms that aim to produce reasonable solutions within a minimal time frame without the guarantee of crafting an optimal solution. Metaheuristics employ population-based approaches. The most well-known metaheuristics are nature-inspired algorithms such as evolutionary algorithms (EAs) [23], these include examples such as, ant colony optimisation (ACO) [24], particle swarm optimisation (PSO) [25], Simulated Annealing (SA), tabu search. These algorithms each harness concepts such as selection, crossover, mutation, adaptation, iterative refinement based on a fitness function. From observing deeper research into metaheuristics, the field of metaheuristics is a highly-developing field, where lots of new metaheuristic algorithms are being created and implemented on optimisation tasks. This survey of PSO [26] showcases the variants of PSO used in real-world applications, their drawbacks, and techniques such as hybridization to enhance PSO while providing further research directions for PSO in complex high-dimensional optimization problems.

Metaheuristics are generally better than simple heuristics [27] as they are not problem-dependent. Metaheuristics can be used in a variety of optimisation problems as they rely on general search principles (exploration/exploitation, balance, adaptation) rather than domain-specific rules, making them highly adaptable to optimisation/scheduling problems and can easily hybridize. These studies by Güven et Al.[28] and alkanhe et Al. [29] demonstrate the usefulness of hybrid metaheuristics for multi-optimisation algorithms and how it outperforms standard metaheuristic algorithms.

From the fundamentals of metaheuristics to the many studies that have been involved in the field of metaheuristics, this shows us that metaheuristics provide more promise than simple heuristics in terms of transfer learning. The ability to generalise enhances the capability of solving multiple unique scheduling tasks.

Be that as it may, this does not call for isolating simple heuristics out of the equation. This is because, in some cases, a simple heuristic may optimally be more efficient if the algorithm has deep/in-depth domain knowledge that can be exploited or the problem is small or has a well-known structure.

2.1.3 Hyper-heuristics

Hyper-heuristics (HH) are higher-level strategies that select or combine already existing heuristics or generate new heuristics or metaheuristics to find the most optimal algorithm to solve the problem. In simpler terms, a hyper-heuristic is a heuristic that decides other heuristics, and finds the best (most efficient) heuristic to solve the problem. According to this study on hyper-heuristics [30], the goal of hyper-heuristics is "to act as effective cross-domain search methodologies i.e. to be applicable not only to instances with different characteristics from a single domain, but also across multiple problem domains" (domain-independent).

There are two types of hyper-heuristic categories: heuristic selection and heuristic generation. Heuristic selection is a methodology to select and choose from an existing heuristic or combine existing heuristics [31]. On the other hand, heuristic generation is methodology to generate new heuristics (from primitive components or existing heuristic search trails) [31]. Furthermore, since heuristics can be constructive or perturbative, we can further categorise the heuristics into a broader classification for HHs: Generation Constructive, Generation Perturbative, Selection Constructive and Selection Perturbative [4]. Burke et Al. also proposed that hyper-heuristics can be considered as a learning algorithm because it receives feedback on the performance on the low-level heuristics from the search process. In addition, they further added a classification on hyper-heuristics based on the feedback obtained from learning the performance of low-level heuristics: Online learning and Offline Learning [31].

- **Online Learning**: Learning takes place as the algorithm is solving a problem instance
- **Offline Learning**: Learning takes place beforehand, from a set of training and testing instances, evolving and improving the model after each trained solution
- **No Learning**: Does not retain information from previous searches to adapt the algorithm

This paper by Yates et Al. [32] proposes a set of methods that explain what a hyper-heuristic is doing during online and offline learning. They use explainability to help demonstrate the effects of the hyper-heuristics such as visualisations, sequence analyses, clustering and utility tracking. The limitations of the proposed paper are that it focuses on selection hyper-heuristics, not on generation hyper-heuristics, and they applied the hyper-heuristics to certain problem domains (e.g. water distribution optimisation), so unfamiliar problem types may reduce heuristic efficiency.

Hyper-heuristics and metaheuristics are both very efficient at finding the optimal solution; however, a well-designed, problem-specific metaheuristic may outperform a hyper-heuristic. The true strength of Hyper-heuristics comes from their generality to be applied to multiple problem domains with minimal change. Furthermore, hyper-heuristics can also use metaheuristics to find the best solution, as HHs are above metaheuristics in the heuristic hierarchy.

After covering the fundamentals of heuristics and hyper-heuristics, this still begs the question of how truly effective hyper-heuristics are at optimisation tasks. This study [33] showcases an effective Hyper Heuristic-based Memetic Algorithm (HHMA) that was applied to a flow-shop scheduling problem (DAFPSP). This paper displays the algorithm containing 7 Low-level heuristics (LLH) and a local search CP-RLS, and it also utilises online learning through EDA sampling. However, the hyper-heuristic lacks generalisation as it is tailored towards the DAFPSP and the LLHs are hand-crafted, therefore it would not adapt well across domains. Another study [34] reviews and showcases the advancements of selection hyper-heuristics after the [4] paper. This study goes into great detail on types

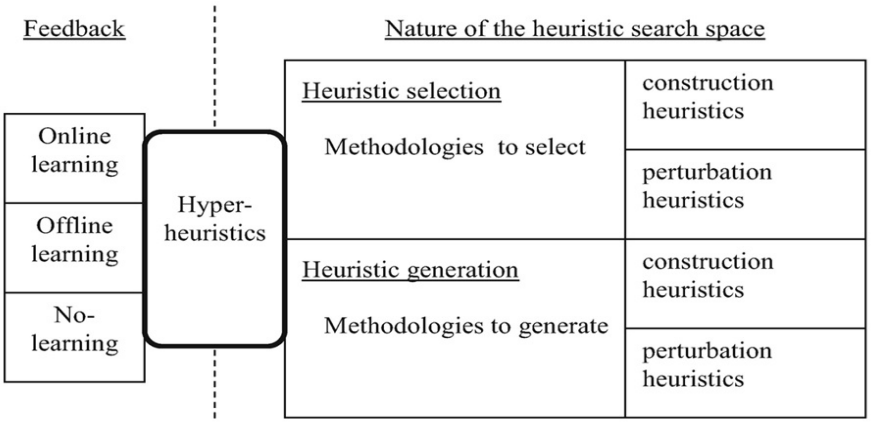

Figure 1: A classification of hyper-heuristics by Burke et Al.[31]

of selection hyper-heuristic variants, scheduling and optimisation problems, and mentions of hyper-heuristic frameworks. For instance, it mentions the HyFlex (Hyper-heuristic Flexible) framework [35] that collects hyper-heuristics used in the CheSC 2011 (Cross-domain Heuristic Search Challenge), for example adapHH [36] and PHunting [37]. The paper also shows a comparative analysis of the different types of hyper-heuristics, displaying the most effective and most efficient algorithms. Other frameworks mentioned are Hyperion [38], ParHyFlex [39] and EvoHyp [40] etc. However, the biggest flaw of this paper and the CHeSC competition is that all the hyper-heuristics did not have a universal metric, which could show the better algorithm but instead ranked through a formula-1 scoring system. This leads to suspicion of bias and one of the biggest criticisms is that it seems to favour optimality over generalization. Another minor limitation of this paper is that it shows a heavy focus on selection HH over generation HH, but it does reference some generation hyper-heuristics.

Further studies covered on hyper-heuristics [41] [42] [43], proves that hyper-heuristics are indeed an effective strategy at handling optimisation tasks.

2.2 Pre-Trained Models and Transfer Learning

There is a subtle nuance between pre-trained models and incorporating transfer learning. The reason for this is because they follow the same principle, where a model is trained on one problem domain (set of problems or data) and then tested on another unique domain. However, transfer learning is a broader view of the scope that aims to improve the model's cross-domain performance through adaptation, while a pre-trained model applies that trained model to the new problem without changing any variables or adapting the model.

After researching extensively into this topic, there are very few studies that reference or explicitly use the term "pre-trained hyper-heuristic". The term "pre-trained" is predominantly associated with machine learning models where it refers to models that have undergone offline training with large datasets (or problems). In contrast, the concept of a pre-trained hyper-heuristic is less formally established in literature, even though similar principles may implicitly exist in works describing offline-trained or offline-evolved hyper-heuristics. In a sense, when a hyper-heuristic obtains information or strategies through prior learning before deployment, it closely aligns with the notion of pre-trained models and transfer learning.

2.2.1 Pre-Trained Models

This paper [44] proposes using a multi-objective based Genetic Algorithm (MOGA) approach for optimising pre-trained text models for text summarization tasks. This study demonstrates that pre-trained models offer significant performance advantages and reduces training resource demands. Importantly, it combines these models with a multi-objective evolutionary algorithm, representing a similar form

of hyper-heuristic control over pre-trained knowledge. However, the work remains limited to a single domain, with no assessment of cross-domain transferability or risks of negative transfer, which are key challenges that pre-trained hyper-heuristics in optimisation must address. It should also be addressed that in this paper, in contrast to my project, the optimiser is not pre-trained, but instead, the genetic algorithm is being trained by another genetic algorithm, which is not what I will be doing for the project.

2.2.2 Transfer Learning

Transfer learning (TL) is a machine learning technique that leverages the knowledge learned from source tasks and transfers it to a different task so it does not require a large amount of data [45] [46]. TL is useful at addressing the issue of insufficient well-labeled data and reducing training cost and data requirements. Another issue that TL addresses is time and computational resources required to train the model are greatly reduced as knowledge from the original domains can be re-used [47].

Some studies have used transfer learning in hyper-heuristics, for example this study [48] investigates transfer learning for an Ant-Based (ACO) Generation Construction Hyper-heuristic. This study claims that transfer learning has helped reduce the computational cost of genetic programming generation constructive hyper-heuristics in previous works, presumably sparking the desire for this study. The study was tested on three domains, Movie Scene Scheduling Problem (MSSP), One-Dimensional Bin Packing Problem (1BBP), Quadratic Assignment Problem (QAP) with their results showing that it effectively reduced the computational costs for all three domains, however, 1BBP suffered a reduction in solution quality. This exposes poor generality as domains require similar characteristics with one another. Another limitation is that the author did not have a method to tackle poor transferability, which would affect the solution quality. Another study [49] uses transfer learning in a Genetic Programming (GP) Hyper-heuristic to solve "Uncertain Capacitated Arc Routing Problem" (UCARP). The study claims that this is the first of its kind, how genetic programming has evolved many UCARP principles but due to the cost of the evolution, it proposes to use transfer learning to reduce it. However, the author acknowledges that none of the transfer methods achieved better results. Instead, it created a better initial population for the GP, achieved faster comparable results for the target domain and allowed observation of blockades that impacted effectiveness. But the author further acknowledges that search degradation effects, for example, an abundance of redundancy or convergence to local optima can cause negative transfer (when prior knowledge harms performance).

2.3 Comparative Analysis for Pre-Trained Hyper-Heuristics

A pre-trained hyper-heuristic would possess offline learning capabilities as it acquires knowledge ahead of time from sampling problem instances or domains. Rather than learning solely during problem-solving, these systems leverage previous experience to infer effective heuristic-selection strategies, embedding this knowledge into a model so it can be immediately deployed. This is necessary because HHs typically begin from scratch when solving a problem. Overcoming this limitation would reduce computational cost and facilitate reusability.

While traditional hyper-heuristics emphasize domain-independence through online learning, pre-trained hyper-heuristics focuses heavily on knowledge-centric optimisation. This paradigm significantly reduces time-to-improvement and enhances scalability for large-scale or time-critical environments. However, it also introduces new questions and challenges associated with generalisation, negative transfer, and model interpretability, particularly when complex machine learning models are used to encode heuristic-selection logic. Therefore, a key open problem is determining what knowledge is safe to transfer and how to robustly adapt it reliably across diverse and potentially unseen optimisation domains.

Ultimately, advancing pre-trained hyper-heuristics requires balancing the efficiency benefits of offline learning with safeguards that maintain adaptability and reliability, consequentially preserving the foundational principle of domain-generality that defines the hyper-heuristic model.

3 Project Specification

3.1 Motivation

Hyper-heuristics is an expanding field in the topic of machine learning. Traditional online-learning hyper-heuristics often begin with no knowledge, due to its online learning capabilities, having to rely on learning during the optimisation process. Pre-trained hyper-heuristics aim to leverage offline knowledge to accelerate optimisation, enhance generalisation and exploit transferable experience while reducing computational cost. The main motivation for doing this project is that this could be a big stepping-stone in the innovation of optimisation algorithms; assuming that pre-trained hyper-heuristics can work well with problems of similar nature, this may allow for optimisation tasks to be solved more efficiently and effectively. The purpose of this paper is to investigate if hyper-heuristics can be pre-trained and if they are better performing than standard online training HHs. Furthermore, this project will also directly address a key open problem identified in literature: determining what specific knowledge (e.g. effective heuristic sequences or selection policies) is safe and beneficial to transfer, and how to do so robustly across domains.

3.2 Aims and Objectives

1. Build a hyper-heuristic and test the hyper-heuristic on certain problem domains.
2. Pre-train the hyper-heuristic and test the new pre-trained hyper-heuristic on the chosen problem domains.
3. Evaluate the scores and compare the performance of the pre-trained hyper-heuristic to the standard online hyper-heuristic.
4. Compare the performance of the pre-trained hyper-heuristic to the HyFlex algorithms.

3.3 Methodology

For this project, I will build the hyper-heuristic using Python, and I will be using Java to adjust the code and retrieve the scores of the hyper-heuristic algorithms from the HyFlex framework. Python is used because of the libraries that grant easy access to machine learning tools for my project and Java is used because the HyFlex framework is done in Java. Therefore, the Python libraries that will be used for the project are Numpy, Pandas, Matplotlib, SciPy, Tensorflow (possibly Keras), scikit-learn, JSON. More libraries may be added depending on whether another feature is required for the project.

3.3.1 Constructing the hyper-heuristic

Building the hyper-heuristic will be a challenge; therefore, I will be using the customHYS framework [50] as assistance and inspiration for building the hyper-heuristic. It is also easier to integrate as it is made in Python. I will also use the HyFlex [35] hyper-heuristics and MatHH [51] hyper-heuristics as inspiration for the design of the hyper-heuristic. However, I will not use MatHH hyper-heuristics as a comparison as there is no substantial compatibility between MatLab and Python.

3.3.2 Pre-training homemade hyper-heuristic

TensorFlow will be used to pre-train the hyper-heuristic. For the sake of compatibility, I will use the same problem domains used in the HyFlex framework, for example Personnel Scheduling, Bin Packing, Flow Shop Scheduling and MAX-SAT problem. These problem domains are provided directly by the HyFlex framework, which includes a standard set of benchmark instances. This project will use these existing instances, which will be divided into training and testing sets. The 'data' for pre-training will therefore be the performance data (e.g., solution quality improvement) generated by applying heuristics to these training instances. Crucially, this project will focus exclusively on single-objective

optimisation tasks, as defined in the CHeSC 2011 competition. This is to provide more visual clarity and distinction between the performance of my hyper-heuristic and the HyFlex algorithms.

3.3.3 Evaluating the scores

In order to properly justify the effectiveness of providing beforehand knowledge to a hyper-heuristic, I will be using various metrics. As this project focuses on single-objective optimisation, effectiveness will be measured by the solution quality (the best objective value found), the time it took for the model to find the solution, and how well it generalises across domains. The results will also be represented in graphs, such as convergence curves, box plots or bar charts to show more visual clarity as to the possible improvement of pre-training a hyper-heuristic.

3.3.4 Comparing pre-trained HH to HyFlex algorithms

To properly compare the results of the proposed pre-trained hyper-heuristic to the ones used in the HyFlex framework, I will need to make an adapter that will be able to retrieve the results from both pre-trained HH and HyFlex algorithms. The Py4J python library will be used to link the bridge ports between Python and Java. The JSON python library will be required for this section. Then the pre-trained hyper-heuristic's scores will be compared with the scores of some of the HyFlex HHs.

3.4 Timeline

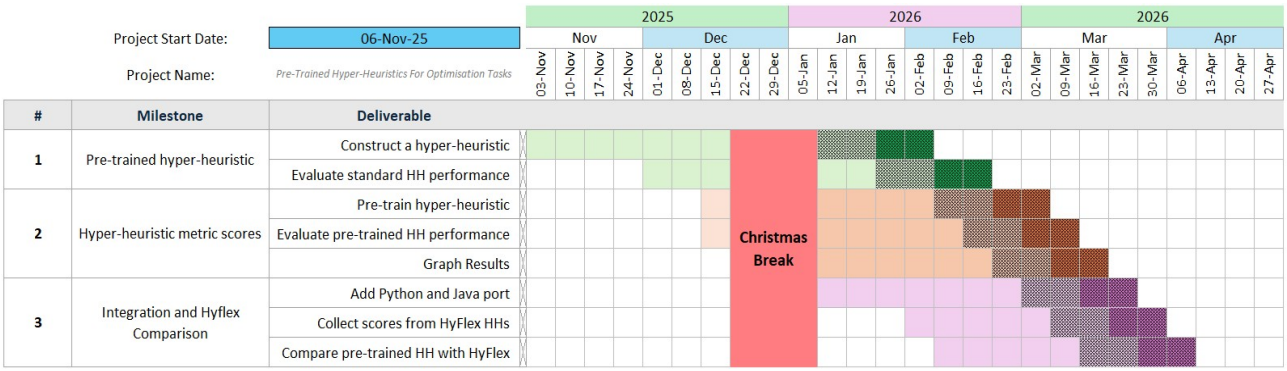

Figure 2: Timeline for the project

KEY:

- The shaded regions represent the contingency time for potential task delays
- The dark coloured slots indicate the latest finish deadline for each task
- Schedule Note: Christmas break is marked as a non-working period, no project work is scheduled

4 Evaluation

4.1 Criteria

The proposed project must obey the following criteria (See Table 1).

4.2 Risks and Resolutions

1. Pre-trained models have the possibility of negative transfer when the pre-trained domain and target domain are too different during offline learning. My proposed solution is fine-tuning the model and use domain-adaptive pretraining to reduce the risk.

Evaluation Criteria	Description
Analysis and model benchmarks	This project should showcase if hyper-heuristics can be pre-trained for optimisation tasks through ablation studies and all tests should be treated fairly by giving the same benchmarks and test parameters.
Model performance	The hyper-heuristic's results should be reliable and accurate, evaluated using metrics such as solution quality, runtime, number of runs, and generalisation. Generalisation will be assessed by comparing the performance of all the tested HHs to each problem domain.
Visualisation and results	The project should produce reproducible results and the results should be easily readable to the user and represented on graphs or diagrams, such as convergence curves, box plots or bar charts.
Ethics	This project will not contain or involve any user data. All models or repositories used must be credited to the appropriate author.

Table 1: Evaluation criterion (incl. description).

2. Pre-trained models may overfit when there is too much domain bias, therefore, I will need to monitor the bias and apply regularization during hyper-heuristic training.

3. A bad heuristic may propagate over many tasks. My proposed solution is to have uncertainty checks, and rollback mechanisms to avoid cascading errors.

5 Conclusion

This literature review has examined the foundational concepts and existing research surrounding hyper-heuristics, pre-trained models, and transfer learning in optimisation tasks. Hyper-heuristics have been shown to provide strong domain-independent search capabilities by intelligently selecting and adapting low-level heuristics, yet they traditionally rely on online learning with no prior knowledge. Pre-trained models, on the other hand, leverage offline-acquired experience to reduce computational requirements and improve performance when deployed on new tasks, though their success depends heavily on domain similarity and transferability.

By analysing related work where transfer learning has been applied to hyper-heuristics, it becomes clear that there is potential in combining the two fields. However, risks such as negative transfer, reduced solution quality in mismatched domains, and lack of generalisation remain open challenges that must be addressed.

This project therefore proposes to explore whether a pre-trained hyper-heuristic can be made and if it can outperform standard online-learning hyper-heuristics by accelerating convergence, improving cross-domain performance, and reducing computational cost. The planned implementation, evaluation approach, and benchmarking against established frameworks seek to provide the first structured analysis of the feasibility and effectiveness of pre-training in a hyper-heuristic context.

Overall, this work aims not only to demonstrate whether pre-trained hyper-heuristics are viable, but also to establish insights that could inform new research directions in general-purpose optimisation where knowledge reuse, scalability, and adaptability are increasingly essential.

References

[1] J. A. Soria-Alcaraz, A. Espinal, and M. A. Sotelo-Figueroa, "Evolvability metric estimation by a parallel perceptron for on-line selection hyper-heuristics," *IEEE Access*, vol. 5, pp. 7055–7063, 2017.

[2] A. Vela, G. H. Valencia-Rivera, J. M. Cruz-Duarte, J. C. Ortiz-Bayliss, and I. Amaya, "Hyper-heuristics and scheduling problems: strategies, application areas, and performance metrics," *IEEE Access*, 2025.

[3] J. Denzinger, M. Fuchs, and M. Fuchs, "High performance atp systems by combining several ai methods," 1996.

[4] E. K. Burke, M. Gendreau, M. Hyde, G. Kendall, G. Ochoa, E. Özcan, and R. Qu, "Hyper-heuristics: A survey of the state of the art," *Journal of the Operational Research Society*, vol. 64, no. 12, pp. 1695–1724, 2013.

[5] P. Cowling, G. Kendall, and E. Soubeiga, "A hyperheuristic approach to scheduling a sales summit, in'selected papers from the third international conference on practice and theory of automated timetabling iii', patat'00," 2001.

[6] J. H. Ong and J. Teo, "Systematic review and open challenges in hyper-heuristics usage on expensive optimization problems with limited number of evaluations," in *2021 IEEE Symposium on Industrial Electronics & Applications (ISIEA)*. IEEE, 2021, pp. 1–6.

[7] D.-Z. Chen, C. Wei, G.-L. Jia, and Z.-H. Hu, "Shortest-path optimization of ship diesel engine disassembly and assembly based on and/or network," *Complexity*, vol. 2020, no. 1, p. 2919615, 2020.

[8] W. Luo, R. Chin, A. Cai, G. Lin, B. Su, and A. Zhang, "A tardiness-augmented approximation scheme for rejection-allowed multiprocessor rescheduling," *Journal of Combinatorial Optimization*, vol. 44, no. 1, pp. 690–722, 2022.

[9] J. Choi and J. Kim, "A tutorial on quantum approximate optimization algorithm (qaoa): Fundamentals and applications," in *2019 international conference on information and communication technology convergence (ICTC)*. IEEE, 2019, pp. 138–142.

[10] K. Kurowski, T. Pecyna, M. Slysz, R. Różycki, G. Waligóra, and J. Wglarz, "Application of quantum approximate optimization algorithm to job shop scheduling problem," *European Journal of Operational Research*, vol. 310, no. 2, pp. 518–528, 2023.

[11] I. Yang, S. A. Burden, R. Rajagopal, S. S. Sastry, and C. J. Tomlin, "Approximation algorithms for optimization of combinatorial dynamical systems," *IEEE Transactions on Automatic Control*, vol. 61, no. 9, pp. 2644–2649, 2016.

[12] C. N. Mavridis and J. S. Baras, "Annealing optimization for progressive learning with stochastic approximation," *IEEE Transactions on Automatic Control*, vol. 68, no. 5, pp. 2862–2874, 2023.

[13] S. G. Johnson, J. Valenti, and F. C. Keil, "Simplicity and complexity preferences in causal explanation: An opponent heuristic account," *Cognitive psychology*, vol. 113, p. 101222, 2019.

[14] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to algorithms*. MIT press, 2022.

[15] J. Kleinberg, *Tardos. Algorithm Design*. Pearson, 2006.

[16] A. Ala, "An approximate algorithm optimization method for the job shop scheduling problem," in *2019 IEEE 6th International Conference on Industrial Engineering and Applications (ICIEA)*, 2019, pp. 893–897.

[17] C. Stone, E. Hart, and B. Paechter, "A cross-domain method for generation of constructive and perturbative heuristics," in *Automated Design of Machine Learning and Search Algorithms*. Springer, 2021, pp. 91–107.

[18] ——, "Automatic generation of constructive heuristics for multiple types of combinatorial optimisation problems with grammatical evolution and geometric graphs," in *Applications of Evolutionary Computation*, K. Sim and P. Kaufmann, Eds. Cham: Springer International Publishing, 2018, pp. 578–593.

[19] J. C. Tay and N. B. Ho, "Evolving dispatching rules using genetic programming for solving multi-objective flexible job-shop problems," *Computers & Industrial Engineering*, vol. 54, no. 3, pp. 453–473, 2008.

[20] C. Paquay, S. Limbourg, and M. Schyns, "A tailored two-phase constructive heuristic for the three-dimensional multiple bin size bin packing problem with transportation constraints," *European Journal of Operational Research*, vol. 267, no. 1, pp. 52–64, 2018.

[21] A. Lodi, S. Martello, and D. Vigo, "Heuristic algorithms for the three-dimensional bin packing problem," *European Journal of Operational Research*, vol. 141, no. 2, pp. 410–420, 2002.

[22] C. Stone, E. Hart, and B. Paechter, "On the synthesis of perturbative heuristics for multiple combinatorial optimisation domains," in *International conference on parallel problem solving from nature*. Springer, 2018, pp. 170–182.

[23] D. Corne and M. A. Lones, *Evolutionary Algorithms*. Cham: Springer International Publishing, 2018, pp. 1–22.

[24] D. P. Mahato, R. S. Singh, A. K. Tripathi, and A. K. Maurya, "On scheduling transactions in a grid processing system considering load through ant colony optimization," *Applied Soft Computing*, vol. 61, pp. 875–891, 2017.

[25] S. A. Alsaidy, A. D. Abbood, and M. A. Sahib, "Heuristic initialization of pso task scheduling algorithm in cloud computing," *Journal of King Saud University-Computer and Information Sciences*, vol. 34, no. 6, pp. 2370–2382, 2022.

[26] T. M. Shami, A. A. El-Saleh, M. Alswaitti, Q. Al-Tashi, M. A. Summakieh, and S. Mirjalili, "Particle swarm optimization: A comprehensive survey," *IEEE Access*, vol. 10, pp. 10 031–10 061, 2022.

[27] A. H. Gandomi, X.-S. Yang, S. Talatahari, and A. H. Alavi, "Metaheuristic algorithms," *Metaheuristic applications in structures and infrastructures*, pp. 1–24, 2013.

[28] A. F. Güven, N. Yörükeren, E. Tag-Eldin, and M. M. Samy, "Multi-objective optimization of an islanded green energy system utilizing sophisticated hybrid metaheuristic approach," *IEEe Access*, vol. 11, pp. 103 044–103 068, 2023.

[29] R. Alkanhel, E.-S. M. El-kenawy, A. A. Abdelhamid, A. Ibrahim, M. A. Alohali, M. Abotaleb, and D. S. Khafaga, "Network intrusion detection based on feature selection and hybrid metaheuristic optimization." *Computers, Materials & Continua*, vol. 74, no. 2, 2023.

[30] J. Swan, P. De Causmaecker, S. Martin, and E. Özcan, *A Re-characterization of Hyper-Heuristics*. Cham: Springer International Publishing, 2018, pp. 75–89.

[31] E. K. Burke, M. R. Hyde, G. Kendall, G. Ochoa, E. Özcan, and J. R. Woodward, *A Classification of Hyper-Heuristic Approaches: Revisited.* Cham: Springer International Publishing, 2019, pp. 453–477.

[32] W. B. Yates, E. C. Keedwell, and A. Kheiri, "Explainable optimisation through online and offline hyper-heuristics," *ACM Transactions on Evolutionary Learning*, vol. 5, no. 2, pp. 1–29, 2025.

[33] H.-B. Song, Y.-H. Yang, J. Lin, and J.-X. Ye, "An effective hyper heuristic-based memetic algorithm for the distributed assembly permutation flow-shop scheduling problem," *Applied Soft Computing*, vol. 135, p. 110022, 2023.

[34] J. H. Drake, A. Kheiri, E. Özcan, and E. K. Burke, "Recent advances in selection hyper-heuristics," *European Journal of Operational Research*, vol. 285, no. 2, pp. 405–428, 2020.

[35] G. Ochoa, M. Hyde, T. Curtois, J. A. Vazquez-Rodriguez, J. Walker, M. Gendreau, G. Kendall, B. McCollum, A. J. Parkes, S. Petrovic, and E. K. Burke, "Hyflex: A benchmark framework for cross-domain heuristic search," in *Evolutionary Computation in Combinatorial Optimization*, J.-K. Hao and M. Middendorf, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 136–147.

[36] M. Mısır, K. Verbeeck, P. De Causmaecker, and G. Vanden Berghe, "An intelligent hyper-heuristic framework for chesc 2011," in *Learning and Intelligent Optimization*, Y. Hamadi and M. Schoenauer, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 461–466.

[37] C. Y. Chan, F. Xue, W. H. Ip, and C. F. Cheung, "A hyper-heuristic inspired by pearl hunting," in *Learning and Intelligent Optimization*, Y. Hamadi and M. Schoenauer, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 349–353.

[38] J. Swan, E. Özcan, and G. Kendall, "Hyperion – a recursive hyper-heuristic framework," in *Learning and Intelligent Optimization*, C. A. C. Coello, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 616–630.

[39] W. Van Onsem and B. Demoen, "Parhyflex: A framework for parallel hyper-heuristics," in *Proceedings of the 25th Benelux Conference on Artificial Intelligence*, vol. 28, 2013, pp. 231–238.

[40] N. Pillay and D. Beckedahl, "Evohyp - a java toolkit for evolutionary algorithm hyper-heuristics," in *2017 IEEE Congress on Evolutionary Computation (CEC)*, 2017, pp. 2706–2713.

[41] F. Zhao, S. Di, J. Cao, J. Tang *et al.*, "A novel cooperative multi-stage hyper-heuristic for combination optimization problems," *Complex System Modeling and Simulation*, vol. 1, no. 2, pp. 91–108, 2021.

[42] P. B. Miranda, R. B. Prudêncio, and G. L. Pappa, "H3ad: A hybrid hyper-heuristic for algorithm design," *Information Sciences*, vol. 414, pp. 340–354, 2017. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0020025516314323

[43] M. Durasević, F. J. Gil Gala, D. Jakobović, and Y. Mei, "Genetic programming as a hyper-heuristic for solving combinatorial optimisation problems," in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, ser. GECCO '25 Companion. New York, NY, USA: Association for Computing Machinery, 2025, p. 1070–1094. [Online]. Available: https://doi.org/10.1145/3712255.3716516

[44] G. B. Mohan, R. P. Kumar, and R. Elakkiya, "Enhancing pre-trained models for text summarization: a multi-objective genetic algorithm optimization approach," *Multimedia Tools and Applications*, pp. 1–17, 2024.

[45] S. J. Pan, I. W. Tsang, J. T. Kwok, and Q. Yang, "Domain adaptation via transfer component analysis," *IEEE Transactions on Neural Networks*, vol. 22, no. 2, pp. 199–210, 2011.

[46] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345–1359, 2010.

[47] S. Niu, Y. Liu, J. Wang, and H. Song, "A decade survey of transfer learning (2010–2020)," *IEEE Transactions on Artificial Intelligence*, vol. 1, no. 2, pp. 151–166, 2020.

[48] E. Singh and N. Pillay, "A study of transfer learning in an ant-based generation construction hyper-heuristic," in *2022 IEEE Congress on Evolutionary Computation (CEC)*, 2022, pp. 1–7.

[49] M. A. Ardeh, Y. Mei, and M. Zhang, "Transfer learning in genetic programming hyper-heuristic for solving uncertain capacitated arc routing problem," in *2019 IEEE Congress on Evolutionary Computation (CEC)*, 2019, pp. 49–56.

[50] J. M. Cruz-Duarte, I. Amaya, J. C. Ortiz-Bayliss, H. Terashima-Marín, and Y. Shi, "Customhys: Customising optimisation metaheuristics via hyper-heuristic search," *SoftwareX*, vol. 12, p. 100628, 2020.

[51] J. M. Cruz-Duarte, J. C. Ortiz-Bayliss, and I. Amaya, "Mathh: A matlab-based hyper-heuristic framework," *SoftwareX*, vol. 18, p. 101047, 2022.

Generative AI Declaration

AI was used resourcefully and responsibly in the making of this literature review:

- Grammatical improvements and shrinking/chopping sentences: `https://gemini.google.com/share/dd58725b852e`